

## **Deliverable 2: Core Functionality Implementation Report**

### **Cross-Language Application Development**

#### **Project Title**

#### **Expense Tracker Application**

#### **Programming Languages**

- Python (Tkinter GUI)
- C++ (Console Application)

### **1. Introduction**

This project aims at developing an Expense Tracker Application in two different programming languages with the same basic functionality. The application was written in two programming languages, Python and C++, and two programming paradigms, the imperative and the functional, to show how the same application can be programmed using different languages and paradigms. Both implementations aimed for an easy method of managing personal expenses by recording, organizing, searching and managing them. The project was aimed at developing workable implementations which meet the functional requirements given in the assignment. The user interfaces are different in both implementations, however both applications have the same basic operations and offer consistent functionality.

### **2. Core Functionality**

The Python and C++ implementations provide the following features:

- Add new expense records.
- View all recorded expenses.
- Search expenses using category or description.
- Delete existing expense records.

- Calculate the total amount of recorded expenses.
- Save expense records to a file.
- Load previously saved expenses automatically when the application starts.

These features are implemented in a graphical interface using the Tkinter library and are available in the Python implementation. The user enters data in text boxes, clicks buttons and reads data from a table that shows expense information.

The C++ implementation is done using a console menu-driven interface and has the same functionality. The options are selected on the menu and data are typed on the keyboard. Both implementations include the same business functionality although they have different user interfaces.

### **3. Language-Specific Features**

The Python implementation illustrates a number of language-specific features. Expense information is stored in a dictionary and lists are used to keep track of collections of expenses. The dynamic typing of Python makes it easy to deal with data, and the Tkinter library makes it easy to quickly develop a graphical desktop application. The implementation of file handling is done through the JSON module, which will allow the data on expenses to be stored on a persistent basis. The C++ program illustrates object-oriented programming principles by creating structures to hold expense information and vectors to hold many expenses. Compile time checking is better with strong typing, leading to more reliable programs. Expenses are stored and retrieved in text files using standard file streams, and the Standard Template Library (STL) makes it easy to manage collections of expenses.

### **4. Code Quality**

Both implementations were clearly and modularly programmed. Individual functions designed for the major tasks of adding expenses, viewing expenses, searching records, deleting expenses, calculating totals, loading data, and saving data are created.

The source code was organized and commented on throughout to make it easily readable and maintainable. The applications were designed in such a way that the user interaction was kept apart from data processing as much as possible, to facilitate the implementation of future improvements.

## 5. Testing and Debugging

Both applications were tested and was found that it was working as expected. Functional testing was carried out by entering various expense records and checking whether the applications were able to show, search, delete and calculate the expenses.

The following test scenarios were completed successfully:

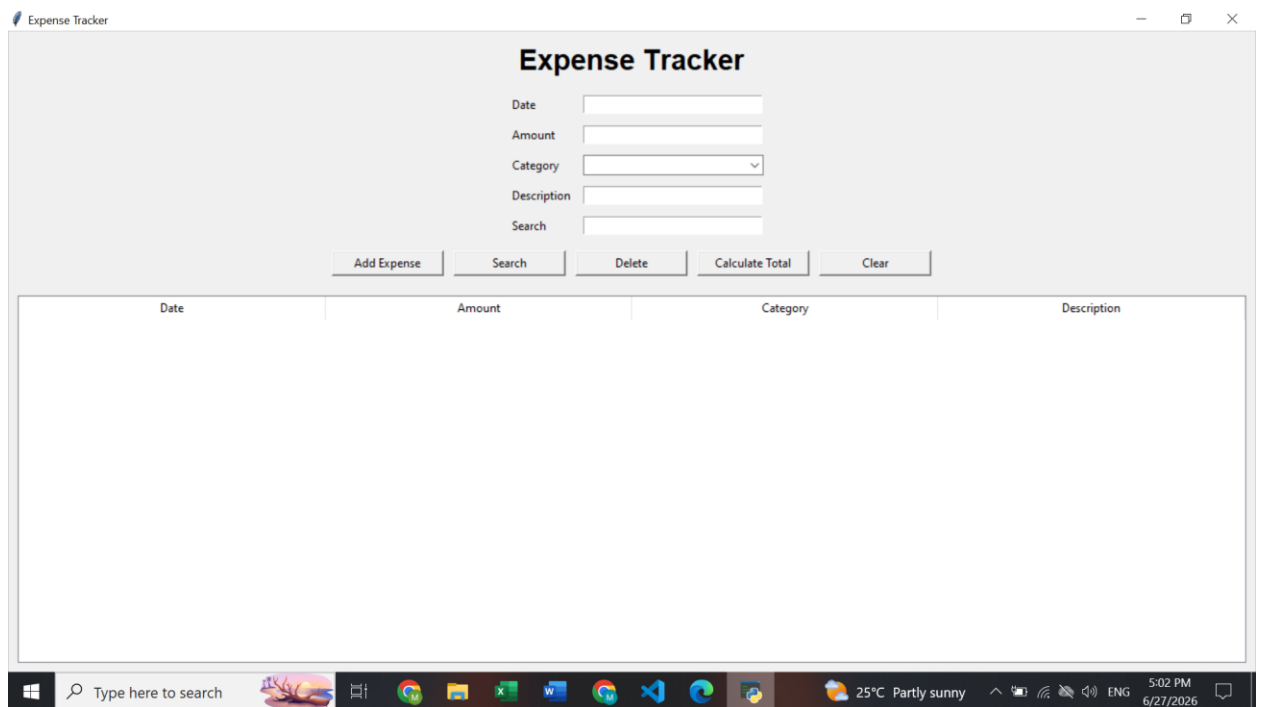
| Test Case                | Result |
|--------------------------|--------|
| Add Expense              | Passed |
| View Expenses            | Passed |
| Search Expenses          | Passed |
| Delete Expense           | Passed |
| Calculate Total Expenses | Passed |
| Save Data                | Passed |
| Load Saved Data          | Passed |
| Invalid User Input       | Passed |

Development was carried out and several debugging activities were carried out. Input validation was enhanced to ensure that invalid inputs were not entered into the program, file handling issues were addressed, and the menu was tested numerous times to ensure that the program always performed the same way. A test of the Python application was also performed to ensure that expense data was still accessible when the application was restarted.

## 6. Screenshots

The following screenshots are included with this submission:

- Python GUI main window.



- Python application displaying expense records.

Expense Tracker

Date

Amount

Category

Description

Search

Add Expense Search Delete Calculate Total Clear

| Date       | Amount | Category  | Description |
|------------|--------|-----------|-------------|
| 2026-05-25 | 999.0  | Bills     | rent        |
| 2026-06-27 | 66.0   | Transport | fare        |

- Python search functionality.

Expense Tracker

Date

Amount

Category

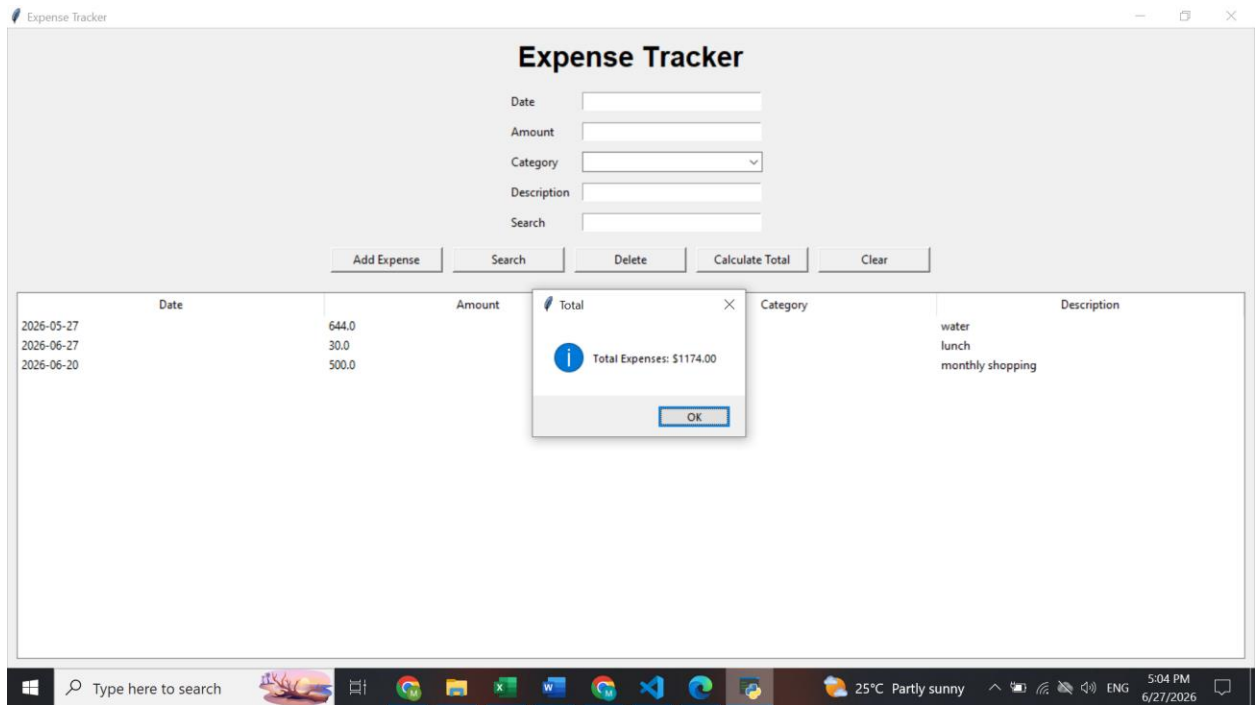
Description

Search

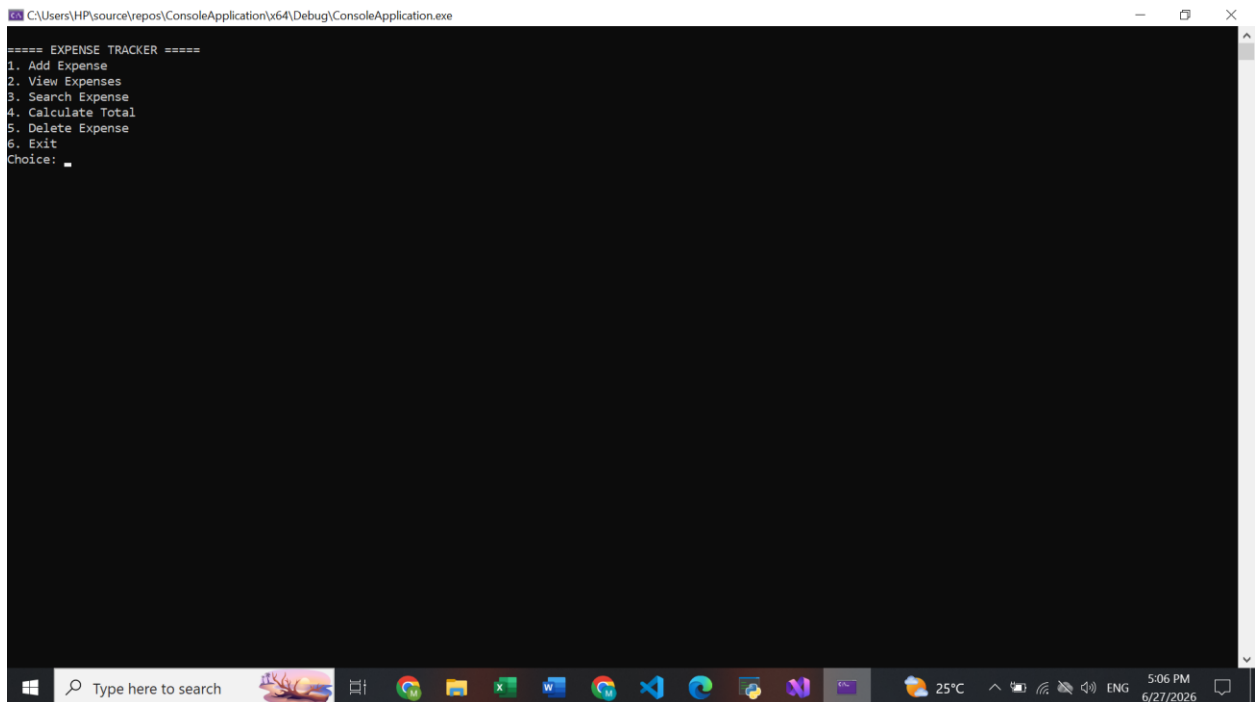
Add Expense Search Delete Calculate Total Clear

| Date       | Amount | Category  | Description |
|------------|--------|-----------|-------------|
| 2026-06-27 | 66.0   | Transport | fare        |

- Python total expense calculation.



- C++ main menu.



- C++ adding an expense.

```
C:\Users\HP\source\repos\ConsoleApplication\Debug\ConsoleApplication.exe

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 1
Date (YYYY-MM-DD): 2026-06-27
Amount: 500
Category: bill
Description: rent
Expense added.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice:
```

- C++ displaying expenses.

```
C:\Users\HP\source\repos\ConsoleApplication\Debug\ConsoleApplication.exe

Amount: 500
Category: bill
Description: rent
Expense added.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 1
Date (YYYY-MM-DD): 2026-05-25
Amount: 50
Category: transport
Description: fare
Expense added.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 2

No Date Amount Category Description
-----
1 2026-06-27 $500.00 bill rent
2 2026-05-25 $50.00 transport fare

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice:
```

- C++ search functionality.

```
C:\Users\HP\source\repos\ConsoleApplication\Debug\ConsoleApplication.exe
Choice: 1
Date (YYYY-MM-DD): 2026-06-26
Amount: 100
Category: food
Description: lunch
Expense added.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 2

No Date Amount Category Description
-----
1 2026-06-27 $500.00 bill rent
2 2026-05-25 $50.00 transport fare
3 2026-06-26 $100.00 food lunch

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 3
Search: bill
2026-06-27 bill $500.00 rent

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice:
```

- C++ total expense calculation.

```
C:\Users\HP\source\repos\ConsoleApplication\Debug\ConsoleApplication.exe
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 2

No Date Amount Category Description
-----
1 2026-06-27 $500.00 bill rent
2 2026-05-25 $50.00 transport fare
3 2026-06-26 $100.00 food lunch

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 3
Search: bill
2026-06-27 bill $500.00 rent

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 4
Total: $650.00

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice:
```

- C++ delete expense demonstration.

```
C:\Users\HP\source\repos\ConsoleApplication\Debug\ConsoleApplication.exe
Total: $650.00

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 5

No  Date      Amount   Category  Description
-----
1  2026-06-27  $500.00  bill      rent
2  2026-05-25  $50.00   transport fare
3  2026-06-26  $100.00  food      lunch
Delete number: 1
Deleted.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 2

No  Date      Amount   Category  Description
-----
1  2026-05-25  $50.00   transport fare
2  2026-06-26  $100.00  food      lunch

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Search Expense
4. Calculate Total
5. Delete Expense
6. Exit
Choice: 
```

## 7. GitHub Repository

The complete project has been uploaded to the team's GitHub repository. The repository contains separate folders for the Python and C++ implementations together with the project documentation and README file.

Repository contents include:

- Python source code.
- C++ source code.
- README.md.
- Documentation.
- Screenshots.

### GitHub Repository Link:

<https://github.com/nadhikari48742/ExpenseTracker>

## **8. Conclusion**

The core functionality of the Expense Tracker Application had been successfully implemented in Deliverable 2 using both Python and C++. We have satisfied all functional requirements for both implementations while demonstrating the differences of the programming languages. Both implementations demonstrate how each programming language works by using a simple GUI with Python's more concise code vs. C++ providing with more stringent type checking and a console-based application development process.

The two programming language implementations of the core functionality provide a good foundation for Deliverable 3. In this phase of the project, we will perform a more detailed comparison of the two programming languages by analyzing their syntax, features, performance, maintainability, and the overall software development experience in relation to the implementation of the core functionality.